ISAILO MAPS

# Guía del proyecto y del código complejo

Explicación sencilla de las pantallas, el mapa, los permisos, el calendario y las partes difíciles de JavaScript y Python.

Instituto San Ignacio de Loyola
Guía para explicar y mantener el proyecto

Edición ampliada: incluye el retorno al mapa general dentro de la misma pestaña, la corrección del mapa oscuro y las reglas actuales del modo Encargado.

# 1. Cómo funciona el proyecto

ISAILO MAPS es una página web. Flask recibe las acciones y arma las pantallas. MySQL guarda usuarios, espacios, solicitudes y eventos. JavaScript conecta los botones con el servidor.

| Parte | En palabras sencillas | Ejemplo |
|---|---|---|
| Pantalla | Lo que la persona ve y usa. | templates/maestro_panel.html |
| Ruta | La dirección que recibe una petición. | /api/mapa/espacios |
| Controlador | Revisa datos y aplica reglas. | controllers/ |
| Modelo | Busca o guarda información. | models/ |
| Base de datos | Donde queda la información. | MySQL |

## El recorrido de una acción

Cuando se pulsa un botón, JavaScript manda los datos a una ruta. La ruta pide al controlador que revise las reglas. El controlador usa un modelo para leer o guardar en MySQL. La respuesta vuelve a la pantalla.

**Botón -> JavaScript -> ruta -> controlador -> modelo -> MySQL -> respuesta**

Las plantillas usan Jinja. Flask procesa sus llaves antes de enviar la página: {{ session['nombre'] }} escribe un dato de sesión y {% if modo_encargado %} decide qué contenido mostrar. Por eso estas páginas deben abrirse desde Flask, no con doble clic.

## Archivos para orientarse

| Archivo o carpeta | Para qué sirve |
|---|---|
| app.py | Inicia Flask y registra los grupos de rutas. |
| routes/ | Recibe páginas y solicitudes del navegador. |
| controllers/ | Aplica las reglas principales de cada acción. |
| models/ | Consulta y modifica MySQL. |
| templates/ y static/ | Pantallas HTML, estilos, imágenes y JavaScript. |
| utils/ | Funciones de apoyo, permisos y relación de salones con zonas. |

# 2. panel.js: cómo entender el archivo grande

static/panel.js coordina muchas pantallas del panel. La idea clave es separar cuatro cosas: guardar los datos, pedirlos, dibujarlos y responder a los clics.

| Nombre o idea | Explicación simple |
|---|---|
| estado | Una libreta temporal con listas y selecciones mientras la página está abierta. |
| api(...) | Envía una petición al servidor y devuelve la respuesta. |
| cargar...() | Pide información y la guarda en estado. |
| pintar...() | Convierte los datos guardados en elementos visibles. |
| showSection(...) | Muestra una pestaña y oculta las demás. |
| esc(...) | Muestra texto como texto, para que un nombre no se interprete como HTML. |

## Ejemplo: mostrar espacios

```
1. cargarEspacios pide /api/espacios.
2. La respuesta se guarda en estado.espacios.
3. pintarEspacios recorre esa lista y dibuja las filas.
4. Si la persona pulsa una fila, se abre su detalle.
```

## El navegador no espera quieto

La conexión con la base puede tardar. fetch y las promesas permiten pedir los datos mientras el navegador sigue activo. then(...) describe qué hacer cuando la respuesta ya llegó. catch(...) permite mostrar un error si algo falla.

```
api('/api/espacios').then(function (respuesta) {
    estado.espacios = respuesta.espacios || [];
    pintarEspacios();
}).catch(mostrarError);
```

## Por qué los clics se escuchan en el contenedor

Muchas filas se crean de nuevo cada vez que cambia una lista. Si el evento se conectara a cada botón viejo, se perdería al redibujar. El panel escucha el clic en un contenedor que permanece y busca dentro el botón pulsado con data-accion y data-id.

**data-accion indica qué hacer; data-id indica sobre qué registro hacerlo. Así el mismo manejador puede responder a muchos botones.**

Para explicar una función: identifica qué recibe, qué cambia, qué pide al servidor y qué pinta al final. No es necesario memorizar las más de dos mil líneas.

# 3. Mapa: de la base de datos a una zona

El mapa combina un plano dibujado y una lista real de espacios. La lista se consulta desde MySQL; utils/espacios_mapa.py conecta el nombre guardado con la zona clicable del plano.

**Actualmente se reconocen estos salones: informática, contabilidad, actos, música, filosofía e informática de primaria. Solo aparecen los espacios que realmente están registrados en la base de datos.**

## Cómo se conecta un salón

| Paso | Qué hace el código |
|---|---|
| 1. Normalizar | Pasa el nombre a minúsculas, quita tildes y reduce espacios repetidos. |
| 2. Buscar | Compara el nombre con las formas conocidas, por ejemplo salón/sala de informática. |
| 3. Leer zona | Usa el valor zona, que debe ser igual al data-espacio del mapa. |
| 4. Mostrar | La pantalla dibuja el espacio y puede seleccionar la zona correspondiente. |

## Ejemplo simplificado

```
'salon de actos': {
    'nombres': ('salon de actos', 'sala de actos'),
    'zona': 'f2-Salon_actos'
}
```

Si un salón existe en MySQL pero todavía no tiene zona dibujada, se deja zona como None. El espacio puede salir en el listado, pero no se puede localizar pulsando el plano. Si todavía no existe en la base, no se debe inventar en este archivo.

## Cómo habla el plano con el panel

El mapa está dentro de un iframe: es una página pequeña dentro del panel. Cuando alguien pulsa una zona, el plano envía al panel un mensaje con el nombre de esa zona. panel.js lo compara con los datos recibidos de la base y selecciona el espacio correcto.

El panel revisa que el mensaje venga del iframe esperado y del mismo sitio. Si la zona no está relacionada con un espacio de la base, muestra un aviso y permite elegirlo desde la lista.

# 4. El mapa oscuro y el botón de regreso

El mapa podía verse negro al entrar rápido a la pestaña. El plano usa zonas que dependen del tamaño visible de su imagen. Si se calculaban mientras el mapa estaba oculto o a mitad de una animación, las zonas quedaban mal ubicadas.

| Ajuste | Por qué ayuda |
| --- | --- |
| El iframe empieza a cargar con el panel. | El plano está más preparado cuando la persona abre la pestaña. |
| La pestaña del mapa se abre sin la animación de entrada. | Evita dejar una transición incompleta cuando se cambia rápido. |
| Al mostrarse, se recalcula el tamaño del plano. | Alinea las zonas clicables con la imagen que se ve en ese momento. |
| Se conserva el tema oscuro. | El arreglo cambia el momento del cálculo, no los colores del diseño. |

## Un solo botón para volver al mapa general

El botón está dentro de la vista de una sección del plano. Al pulsarlo, esconde los pisos o detalles abiertos, muestra otra vez el plano general y limpia la selección. No lleva al inicio del sitio ni saca a la persona de la pestaña Mapa.

```
Volver al mapa general
-> cerrar detalle de sección
-> mostrar el plano general
-> limpiar la selección activa
```

**En el plano general no hace falta un botón de regreso: ya estamos en el inicio del mapa. Por eso se quitaron los botones duplicados que volvían al panel y se dejó el botón interno del mapa.**

## Recorrido virtual

El botón aparece como próximamente y está desactivado. Es solo el espacio reservado para una función futura; actualmente no abre otra página ni inicia un recorrido.

# 5. Modo Docente y modo Encargado

Una misma cuenta docente puede funcionar en dos modos cuando tiene espacios asignados. utils/modo_docente.py comprueba los espacios actuales y el modo guardado en la sesión.

| Modo | Qué puede hacer |
|---|---|
| Docente | Consulta espacios, puede pedir uno y revisa sus propias solicitudes. |
| Encargado | Consulta los espacios asignados, revisa solicitudes recibidas y organiza sus eventos. No pide esos salones. |

## La regla se verifica en dos lugares

La plantilla y panel.js ocultan botones que no aplican para el modo actual. Eso hace la pantalla más clara, pero esconder un botón no es suficiente para proteger una acción.

**Por eso las rutas del servidor también comprueban el modo antes de aceptar una solicitud o permitir cambios. Si un encargado intenta mandar la petición directamente, el servidor responde que no tiene permiso.**

## Qué ocurre si cambian los espacios asignados

modo_actual() consulta los espacios asociados a la cuenta. Si ya no hay ninguno, fija el modo en Docente. Así la sesión no conserva el modo Encargado cuando ya no se cumple la condición.

```
Cuenta docente
    |
    +-- tiene espacios asignados -> puede usar Encargado
    |
    +-- no tiene espacios -> vuelve a Docente
```

En el panel, data-modo comunica el modo actual a panel.js. Este valor ayuda a decidir qué acciones mostrar. Las reglas finales permanecen en Python, dentro de las rutas.

# 6. Calendario, notificaciones y eventos

| Función | Cómo funciona |
|---|---|
| Calendario cultural de inicio | Consulta sin iniciar sesión los eventos de tipo Cultural. Solo muestra información; no incluye acciones para cambiarla. |
| Calendario del panel | Muestra disponibilidad de espacios según el modo y los datos consultados al servidor. |
| Notificaciones propias | Muestra las solicitudes hechas por el docente y su estado. |
| Notificaciones recibidas | En modo Encargado muestra solicitudes de sus espacios. |
| Gestión de eventos | El encargado crea o modifica eventos asociados a sus espacios permitidos. |

## Calendario público: lectura desde la base

La dirección /api/publico/eventos-culturales no pide sesión. La ruta pide al modelo los eventos con tipo Cultural y devuelve una lista en formato JSON. static/js/calendario-publico.js la recibe y dibuja el calendario. Los botones permiten consultar otro mes, no editar eventos.

```
Navegador -> /api/publico/eventos-culturales
Ruta -> busca solo tipo Cultural
Modelo -> lee eventos de MySQL
JavaScript -> pinta el calendario
```

## Notificaciones: diseño y datos

Los archivos HTML de notificaciones definen la apariencia. La página incrustada carga esos archivos y panel.js los conecta con solicitudes reales según el modo. Una notificación no es el dato original: el dato está en la base y la pantalla lo presenta.

## Protección de eventos

Las rutas de eventos vuelven a verificar el modo Encargado. También comprueban que los espacios elegidos pertenezcan al docente. Al editar o borrar, se comprueba que el evento esté relacionado únicamente con espacios permitidos.

# 7. Solicitudes y horarios cruzados

Una solicitud es una petición para usar un salón durante un día y unas horas. Mientras está pendiente todavía necesita revisión. Al aprobarse, ocupa ese intervalo.

## La regla para saber si dos horarios chocan es:

**inicio nuevo < fin ocupado Y fin nuevo > inicio ocupado**

Dicho fácil: hay cruce si el nuevo horario empieza antes de que termine el ocupado y termina después de que el ocupado ya comenzó. Si una actividad termina justo cuando empieza la siguiente, no se cruzan.

| Horario ocupado | Nuevo horario | Resultado |
|---|---|---|
| 10:00 a 11:00 | 10:30 a 11:30 | Se cruzan. |
| 10:00 a 11:00 | 11:00 a 12:00 | No se cruzan; uno empieza cuando termina el otro. |
| 10:00 a 11:00 | 09:00 a 10:30 | Se cruzan. |

## Dónde se revisa

La pantalla puede advertir que una hora parece libre, pero la regla importante está en el servidor. El servidor vuelve a consultar la ocupación antes de guardar o aprobar. Así no depende de lo que el navegador muestre.

En modo Encargado, la lista de solicitudes se limita a los espacios asignados. En modo Docente se filtra a las solicitudes propias. El cambio de estado se envía a una ruta y Python revisa si puede aprobarse.

**Idea clave: el navegador ayuda a usar el sistema; el servidor decide si la acción realmente está permitida.**

# 8. Lo más difícil de Python, explicado fácil

| Concepto | Explicación para el equipo |
|---|---|
| Decorador de acceso | Una puerta delante de la ruta. Comprueba sesión o rol antes de ejecutar la función. |
| Sesión | Una ficha que recuerda quién inició sesión y cuál es su rol o modo. |
| Blueprint | Un grupo ordenado de rutas, como las de eventos o espacios. |
| JSON | Una forma sencilla de enviar datos entre JavaScript y Python. |
| Consulta parametrizada | La base recibe valores por separado del texto SQL. Ayuda a consultar con seguridad. |
| Commit / rollback | Commit confirma un cambio. Rollback deshace los cambios si algo falla. |
| Foreign key / relación | Une registros, por ejemplo un evento con sus espacios. |

## Cuando algo falla, revisar en este orden

| Problema | Primera revisión |
|---|---|
| La página no abre | Que Flask esté iniciado y la dirección sea correcta. |
| El listado está vacío | Que MySQL tenga registros y la ruta devuelva datos. |
| Un botón no aparece | Rol, modo actual y condiciones de la plantilla. |
| No guarda una acción | Respuesta de la ruta y regla del controlador. |
| El mapa se ve oscuro | Carga del iframe y nuevo cálculo al mostrar la pestaña. |
| No deja pedir el salón | Modo actual, disponibilidad y regla de cruce de horarios. |

## Para iniciar el proyecto

```
1. Enciende MySQL.
2. Instala dependencias: pip install -r requirements.txt
3. Inicia Flask: python app.py
4. Abre: http://127.0.0.1:5000/
```

# 9. Guion simple para explicar el proyecto

**“ISAILO MAPS es una aplicación del instituto para consultar espacios, eventos y horarios. Flask recibe las acciones y MySQL guarda la información. Cuando la persona usa un botón, JavaScript envía la petición al servidor; Python comprueba los permisos y las reglas antes de responder.”**

“El mapa lee los espacios de la base de datos y relaciona algunos nombres con zonas del plano. Si una zona todavía no se conoce, el espacio puede seguir en la lista y se completa el mapa después.”

“El docente puede trabajar como Docente o como Encargado si tiene espacios asignados. El modo Encargado no pide espacios: gestiona sus espacios y revisa las solicitudes recibidas. El servidor vuelve a comprobar esos permisos.”

“La parte más extensa es panel.js. Este archivo pide datos, los guarda mientras la página está abierta, dibuja las listas y escucha los botones. Para seguir una acción, miramos desde el botón hasta la respuesta del servidor.”

“El calendario cultural de inicio solo consulta los eventos culturales de la base de datos. No permite modificarlos y no pide iniciar sesión. El mapa mantiene su tema oscuro y el único botón de regreso dentro del plano vuelve al mapa general sin salir del panel.”

## Palabras para recordar

| Palabra | Forma fácil de decirlo |
|---|---|
| Ruta | La dirección que recibe una acción. |
| Controlador | El que revisa las reglas. |
| Modelo | El que busca o guarda datos. |
| Promesa | Una respuesta que llega después. |
| Plantilla | Una página que Flask completa antes de mostrarla. |
| Iframe | Una página pequeña que se muestra dentro de otra. |

**La mejor forma de explicar código complejo es contar qué información entra, qué regla se aplica y qué ve la persona al final.**